



Department of Physics
Jadavpur University

Core-14 (Practical): Computational Physics
Course Code: SC/PHYS/UG/MAJOR/PR/41/414

Course Instructor name:
Prof. Debasish Lohar

Name: ARPAN PURKAIT

Class Roll: 002320701045

Undergraduate-IV

VIIth Semester

ACADEMIC SESSION: 2026-2027

Table Of Contents:

Serial	Chapter	Page No.
1.	Linear Congruential Method (LCM)	2
2.	Built-in Fortran Random Number Generator	8
3.	One-Dimensional Random Walk Problem	13

Chapter – 1

Linear Congruential Method (LCM)

The **Linear Congruential Method (LCM)** is one of the oldest and most widely used algorithms for generating a sequence of *pseudo-random numbers*. Introduced by D. H. Lehmer in 1951, the method relies on a simple linear recurrence relation to yield integers in a specified range.

Because the algorithm is deterministic, the generated numbers are not strictly random in a physical sense (hence the term *pseudo-random*); however, they pass standard statistical tests for uniformity when parameters are chosen correctly.

The sequence of pseudo-random integers $\{X_n\}$ is generated using the following iterative relation:

$$X_{n+1} = (a \cdot X_n + c) \bmod m \quad (1)$$

Where the initial starting value, X_0 is known as Seed. To convert these integer values into standard uniform continuous random variables $U_n \in [0,1)$, we divide X_n by the modulus m :

$$U_n = \frac{X_n}{m} \quad (2)$$

Since X_n can only take values $0, 1, 2, \dots, m-1$, eventually a value must repeat. Once a value repeats, the entire sequence becomes periodic.

The maximum possible period is therefore

$$T_{\max} = m$$

Depending on the value of the increment c , the Linear Congruential Method is divided into two primary types:

1. Mixed Congruential Method ($c > 0$)
2. Multiplicative Congruential Method ($c = 0$)

For a mixed LCG ($c \neq 0$), the sequence achieves the maximum period m , if the **Hull–Dobell conditions** are satisfied:

1. c and m are relatively prime:

$$\gcd(c, m) = 1$$

2. Every prime factor of m divides $a - 1$.
3. If m is divisible by 4, then $a - 1$ must also be divisible by 4.

Note: This reproducibility is actually very useful in **Monte Carlo simulations**, because the same seed allows you to reproduce the same numerical experiment.

Problem 1:

Generate N number of normalized random numbers using Linear Congruential Method (LCM) with the following parameters –

Case	a	c	m	Seed	N
1	57	1	256	10	1280
2	57	1	32768	10	1280

To evaluate the quality and randomness of the generated sequence, perform the following computational tests: (i) histogram, (ii) sample mean and variance, (iii) serial plot test and (iv) period test. Discuss the results and comment on the overall quality of the random number generator.

Solution:

Two set of random numbers are generated by using the parameters given in the above table.

```

1  ! Pseudo-Random Number Generator (PRNG)
2  ! Linear Congruential Method (LCM)
3  program random01_main
4      implicit none
5      integer, parameter :: number_of_values = 1280
6      integer, parameter :: a = 57
7      integer, parameter :: c = 1
8      integer, parameter :: seed = 10
9      real :: r_all(number_of_values)
10     integer, dimension(2) :: m = [256, 32768]
11     integer :: i, j
12     character(len=20) :: filename
13     do j = 1, 2
14         write(filename, '(A,I0,A)') "random_m", m(j), ".dat"
15         open(unit=10, file=trim(filename), status="unknown")
16         call generate_lcm_rand(seed, a, c, m(j), number_of_values, r_all)
17         do i = 1, number_of_values
18             write(10, *) r_all(i)
19         end do
20         close(10)
21     end do
22 end program random01_main
23
24 subroutine generate_lcm_rand(seed, a, c, m, number_of_values, r_all)
25     implicit none
26     integer, intent(in) :: seed, a, c, m, number_of_values
27     real :: r_all(number_of_values)
28     integer :: i
29     integer :: r
30     r = seed
31     do i = 1, number_of_values
32         r_all(i) = real(r)
33         r = mod(a * r + c, m)
34     end do
35     r_all = r_all / real(m)
36 end subroutine generate_lcm_rand

```

(i) Histogram Test:

It is a simple statistical test used to check whether a random number generator produces numbers with the expected probability distribution.

For a generator intended to produce uniform random numbers in the interval

$$0 \leq x < 1,$$

we divide the interval into N_b equal bins. If N random numbers are generated, each bin should contain approximately $N_{\text{expected}} = \frac{N}{N_b}$ numbers.

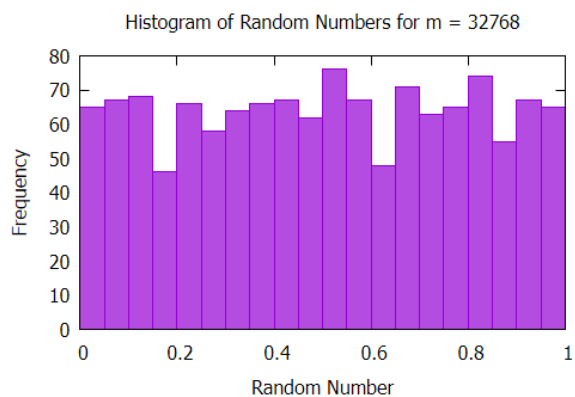
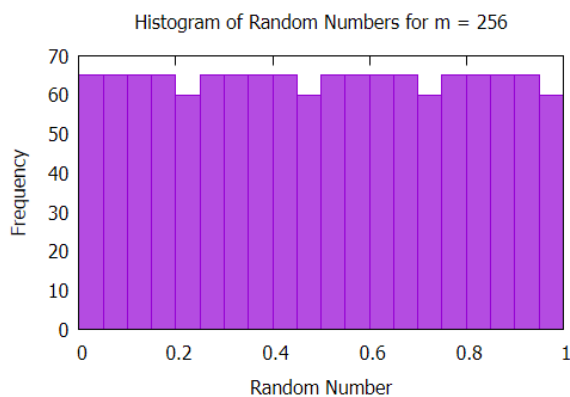
Uniform histogram: suggests that the numbers are uniformly distributed.

Large variations between bins: may indicate non-uniformity or correlations in the generator.

```

1  set terminal pngcairo size 1000,700
2  set output "random_m_32768_histogram.png"
3
4  set title "Histogram of Random Numbers for m = 32768"
5  set xlabel "Random Number"
6  set ylabel "Frequency"
7
8  set xrange [0:1]
9  set boxwidth 0.05
10 set style fill solid 0.7
11
12 bin_width = 0.05
13 bin(x,width) = width * floor(x/width) + width/2.0
14
15 plot "random_m32768.dat" using (bin($1,bin_width)):(1.0) \
16      smooth freq with boxes notitle

```



Comment: For $m = 256$ (Left), the bars are approximately the same height, around 65. This means the generated numbers are **quite uniformly distributed** over $0 \leq x < 1$. For $m = 32768$ (Right), the bars fluctuate more strongly, roughly from 45 to 77. This indicates that the distribution is **less uniform visually** than for $m = 256$. Some intervals contain noticeably more numbers than others. However, this **does not necessarily mean $m = 32768$ is a worse random-number generator**. Histogram fluctuations naturally occur because of finite sample size.

(ii) Sample Mean and Variance Test:

For a given random sample of size N , we estimate the population moments using sample statistics:

- Sample Mean ($\bar{X}$): $\bar{X} = \frac{1}{N} \sum_{i=1}^N X_i$
- Variance (s^2): $s^2 = \frac{1}{N} \sum_{i=1}^N (X_i - \bar{X})^2$

For our dataset we calculated sample mean and variance by this Fortran code,

```

1  program random01_mean
2      implicit none
3      integer, parameter :: number_of_values = 1280
4      real :: r_all(number_of_values)
5      integer :: i
6      real :: mean_value
7      real :: variance
8
9      open(unit=10, file="random_m256.dat", status="old")
10     read(10, *) (r_all(i), i = 1, number_of_values)
11     close(10)
12     mean_value = sum(r_all) / real(number_of_values)
13     print *, "Mean of random numbers (m=256): ", mean_value
14     variance = sum((r_all - mean_value)**2) / real(number_of_values)
15     print *, "Variance of random numbers (m=256): ", variance
16 end program random01_mean

```

The output is :

```

1  Mean of random numbers (m=256):    0.498046875
2  Variance of random numbers (m=256):  8.33305344E-02
3  Mean of random numbers (m=32768):   0.503021240
4  Variance of random numbers (m=32768): 8.28429013E-02

```

Comment: Both datasets pass the mean and variance test. The observed deviation of mean for both $m = 256$ and $m = 32768$ are within 0.002. The observed variance is also close to $1/12$.

(iii) Serial Plot test:

The serial plot test checks whether successive random numbers are independent of each other. Suppose your generator produces $R_1, R_2, R_3, \dots, R_N$ where $0 \leq R_i < 1$.

We construct pairs of successive numbers:

$$(R_1, R_2), (R_2, R_3), (R_3, R_4), \dots$$

and plot : R_n vs. R_{n+1} as a scatter plot.

For a good random number generator, the points should be randomly scattered throughout the unit square:

$$0 \leq R_n < 1, 0 \leq R_{n+1} < 1.$$

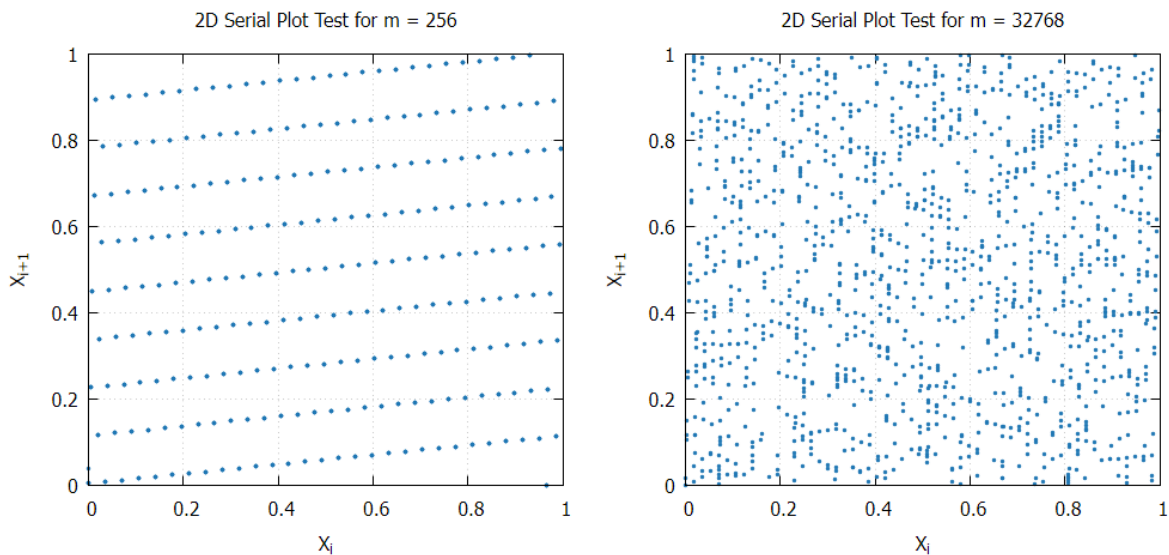
There should be no visible patterns like - line, curve, clustering, or empty regions.

An LCG can pass a simple histogram test and still have correlations between successive numbers. For example, *pseudo-random numbers* can produce points that form parallel lines or lattice-like structures in the R_n - R_{n+1} plane.

```

1  set terminal pngcairo size 500,500
2  set output "random_m_32768_serial.png"
3
4  set title "2D Serial Plot Test for m = 32768"
5  set xlabel "X_{i}"
6  set ylabel "X_{i+1}"
7  set xrange [0:1]
8  set yrange [0:1]
9  set grid
10
11 prev = 0.0
12 lag1(x) = (tmp = prev, prev = x, tmp)
13
14 plot "random_m32768.dat" using (lag1($1)):1 with points pt 7 ps 0.4 lc rgb "#1f77b4" notitle

```



Comment: We can see that, $m = 256$ can generate at most 256 distinct values. When plotting 1280 points, the generator wraps around its period 5 times, overlaying the exact same points again. Where, $m = 32768$ has 128 times more possible discrete values than 256. So, the lattice is incomplete (or not clearly visible). But from here we can conclude that our pseudo random number generator failed the serial test completely.

(iv) Period Test:

For a Linear Congruential Generator, the period is the number of iterations required for the sequence to return to its initial seed X_0 .

In other words, generate

$$X_0, X_1, X_2, \dots$$

until

$$X_n = X_0$$

for the first time. Then n is the period T .

```

1      write(*, *) "Periods for different moduli:"
2      do j = 1, 2
3          do i = 2, number_of_values
4              if (r_all(i, j) == r_all(1, j)) then
5                  period = i - 1
6                  write(*, *) "Modulus ", m(j), ": ", period
7                  exit
8              end if
9              if (i == number_of_values) then
10                 write(*, *) "Modulus ", m(j), ": No period found within ", number_of_values,
11                                     \ " values."
12             end if
13         end do
14     end do

```

The output is:

```

1      Periods for different moduli:
2      Modulus      256 :      256
3      Modulus     32768 : No period found within      1280 values.

```

Comment: For $m = 256$ we get period (T) = 256. This is a **full-period generator**, because $T = m = 256$. The Hull–Dobell conditions are satisfied: $\gcd(c, m) = \gcd(1, 256) = 1$, 256 has only prime factor 2, and $2 \mid (57 - 1) = 56$ and also $4 \mid 56$ so $T = 256$

But for $m = 32768$, we generated only $N = 1280$ random numbers and obtained no period within 1280 values. This does not mean that the generator has no period. It means only that: $T > 1280$. We did not encounter the initial seed again within the 1280 generated values. For $m = 32768 = 2^{15}$, the Hull–Dobell conditions are again satisfied: $\gcd(1, 32768) = 1$ and $2 \mid 56$, $4 \mid 56$. Therefore the theoretical period is $T = 32768$

Chapter – 2

Built-in Fortran Random Number Generator

Modern Fortran has a built-in pseudorandom number generator (PRNG) through the intrinsic procedure named *srand(seed)*. General work-flow is -

```

1  ...
2  call srand(seed)
3  random_number = rand()
4  ...

```

Problem 2:

Generate N number of normalized random numbers using build-in Fortran random number generator with the following parameters –

Case	Seed	N
1	10	1280
2	12	1280

To evaluate the quality and randomness of the generated sequence, perform the following computational tests: (i) histogram, (ii) sample mean and variance, (iii) serial plot test and (iv) period test. Discuss the results and comment on the overall quality of the random number generator.

Solution:

I generated two files with sequences of random numbers for two different seed values by using Fortran inbuilt random number generator.

```

1  program random02_main
2      implicit none
3      integer, parameter :: number_of_values = 1280
4      integer :: seed(2) = [10, 12]
5      character(len=20) :: filename
6      integer :: i, j
7      do i = 1,2
8          call srand(seed(i))
9          write(filename, '(A,I0,A)') "random_seed", seed(i), ".dat"
10         open(unit=10, file=trim(filename), status="unknown")
11         do j = 1, number_of_values
12             write(10,*) rand()
13         end do
14         close(10)
15     end do
16 end program random02_main

```

(i) Histogram Test:

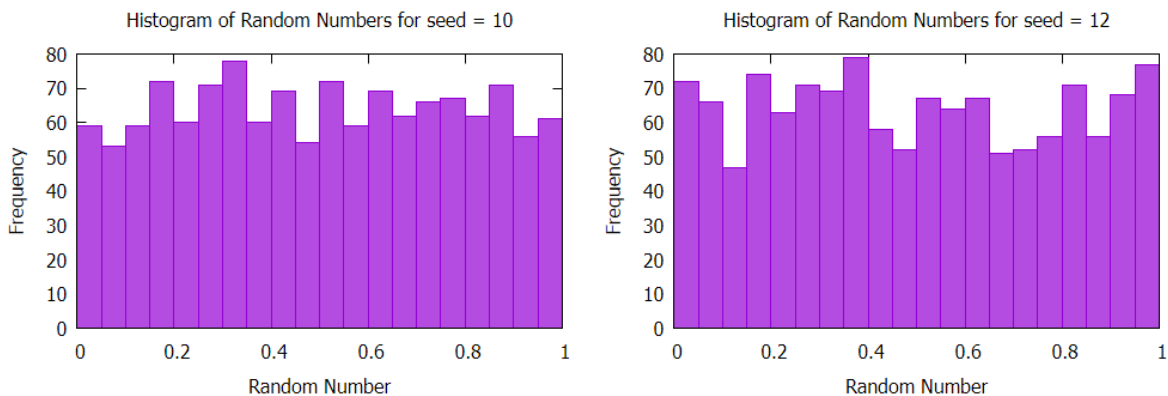
GNU plot code to generate histogram of our random number sequences. Here bin width chosen is 0.05. Number of bins = 20

```

1  set terminal pngcairo size 500,350
2  set output "random_seed12_histogram.png"
3
4  set title "Histogram of Random Numbers for seed = 12"
5  set xlabel "Random Number"
6  set ylabel "Frequency"
7
8  set xrange [0:1]
9  set boxwidth 0.05
10 set style fill solid 0.7
11
12 bin_width = 0.05
13 bin(x,width) = width * floor(x/width) + width/2.0
14
15 plot "random_seed12.dat" using (bin($1,bin_width)):(1.0) \
16     smooth freq with boxes notitle

```

The output is:



Comment: Both plots depict pseudo-random number generators outputting values continuously uniform across the range [0,1]. Changing the seed alters the deterministic sequence generated, leading to different localized statistical fluctuations. But overall macro-level uniformity remains intact.

(ii) Sample Mean and Variance Test:

Fortran code to calculate Sample Mean and Variance of the generated random number sequences for different seeds.

```
1  program random01_mean
2      implicit none
3      integer, parameter :: number_of_values = 1280
4      real :: r_all(number_of_values)
5      integer :: i
6      real :: mean_value
7      real :: variance
8
9      open(unit=10, file="random_seed10.dat", status="old")
10     read(10, *) (r_all(i), i = 1, number_of_values)
11     close(10)
12     mean_value = sum(r_all) / real(number_of_values)
13     print *, "Mean of random numbers (seed=10): ", mean_value
14     variance = sum((r_all - mean_value)**2) / real(number_of_values)
15     print *, "Variance of random numbers (seed=10): ", variance
16 end program random01_mean
```

The mean and variance for different seeds are-

```
1  Mean of random numbers (seed=10):  0.501938760
2  Variance of random numbers (seed=10):  8.03999007E-02
3
4  Mean of random numbers (seed=12):  0.496389329
5  Variance of random numbers (seed=12):  8.62128213E-02
```

Comment: Both datasets pass the mean and variance test. The observed deviation of mean for both seed = 10 and seed = 12 are within 0.005. The observed variance is also close to 1/12.

(iii) Serial Plot test:

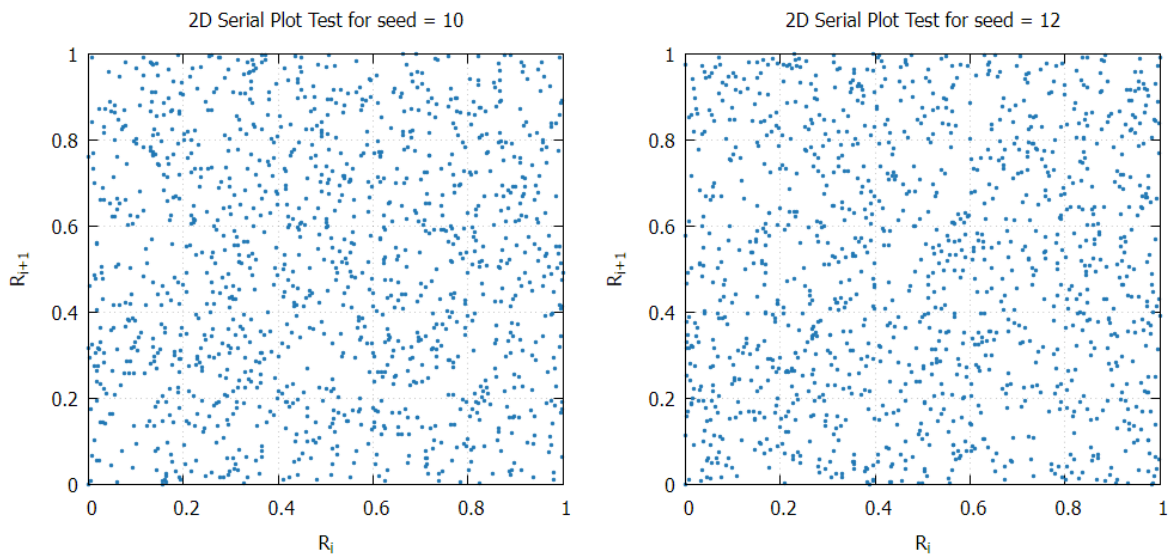
GNUPLOT code to generate serial plots of the generated random number sequences for different seeds.

```

1  set terminal pngcairo size 500,500
2  set output "random_seed12_serial.png"
3
4  set title "2D Serial Plot Test for seed = 12"
5  set xlabel "R_{i}"
6  set ylabel "R_{i+1}"
7  set xrange [0:1]
8  set yrange [0:1]
9  set grid
10
11  prev = 0.0
12  lag1(x) = (tmp = prev, prev = x, tmp)
13
14  plot "random_seed12.dat" using (lag1($1)):1 with points pt 7 ps 0.4 lc rgb "#1f77b4" n

```

The outputs for two different seed values are –



Comment: In both cases (seed = 10 and seed = 12), the points are scattered uniformly across the unit square $[0,1] \times [0,1]$ without forming noticeable lines, hyperplanes, clusters, or diagonal bands. The 2D density remains uniform, confirming that knowing a generated value R_i provides no obvious predictive information about the subsequent value R_{i+1} . As expected, changing the seed alters the exact point trajectories while maintaining the macro statistical property of spatial uniformity across the Phase Space.

(iv) Period Test:

Fortran code to detect any periods present in our random number sequence -

```

1  program random01_mean
2      implicit none
3      integer, parameter :: number_of_values = 1280
4      real :: r_all(number_of_values)
5      integer :: i, period
6
7      open(unit=10, file="random_seed10.dat", status="old")
8      read(10, *) (r_all(i), i = 1, number_of_values)
9      close(10)
10
11     period = 0
12     do i = 2, number_of_values
13         if (r_all(i) == r_all(1)) then
14             period = i - 1
15             exit
16         end if
17     end do
18     print *, "Period of random numbers (seed=10): ", period
19 end program random01_mean

```

The results are –

```

1  Period of random numbers (seed=10):          0
2  Period of random numbers (seed=12):          0

```

Comment: No period found within the range of $N = 1280$. But Fortran's built-in random number generator has a period and an extraordinarily long one.

GNU Fortran (gfortran): Uses the **xoshiro256** algorithm** in modern releases, which has a period of:

$$P = 2^{256} - 1 \approx 1.15 \times 10^{77}$$

(Older gfortran use **Marsaglia's KISS algorithm**, with a period of $2^{123} \approx 10^{37}$.)

Intel Fortran (ifort / ifx): Uses **multiple combined linear congruential generators** with a period of approximately 10^{18} .

To put $2^{256} - 1$ into perspective: if modern CPUs generate 100 billion random numbers per second, it will take around 3.6×10^{59} years to complete a single period cycle.

Chapter – 3

One-Dimensional Random Walk Problem

Problem 3:

Write a program to simulate a **1D Random Walk** for three independent walkers over a set number of steps.

Each walker starts at position 0. At each step, a uniform random number in the range [0,1) is generated: If the random number is **less than 0.5**, the walker moves one step to the **left** (position decreases by 1) and if the random number is **greater than or equal to 0.5**, the walker moves one step to the **right** (position increases by 1).

Solution: Fortran program to generate data of three random walkers by using rand() fn -

```

1  program main
2      implicit none
3      integer, parameter :: walker_number = 3
4      integer, parameter :: random_numbers = 1000
5      integer :: seed(walker_number) = [100, 1, 103985]
6      integer :: i, j
7      integer :: step
8      real :: random_walk_matrix(random_numbers, (2*walker_number+1))
9      random_walk_matrix = 0.0
10
11     do i = 1, random_numbers
12         random_walk_matrix(i,1) = i
13     enddo
14
15     do i = 1, walker_number
16         call srand(seed(i))
17         step = 0
18
19         do j = 1, random_numbers
20             random_walk_matrix(j,2*i) = rand()
21             if (random_walk_matrix(j,2*i) < 0.5) then
22                 step = step - 1
23             else
24                 step = step + 1
25             end if
26             random_walk_matrix(j,2*i+1) = step
27         enddo
28     enddo
29
30     open(10, file='random_walk_data.txt', status='unknown')
31     do i = 1, random_numbers
32         write(10,*) random_walk_matrix(i,:)
33     enddo
34     close(10)
35     call execute_command_line("gnuplot random03.gnu")
36 end program main

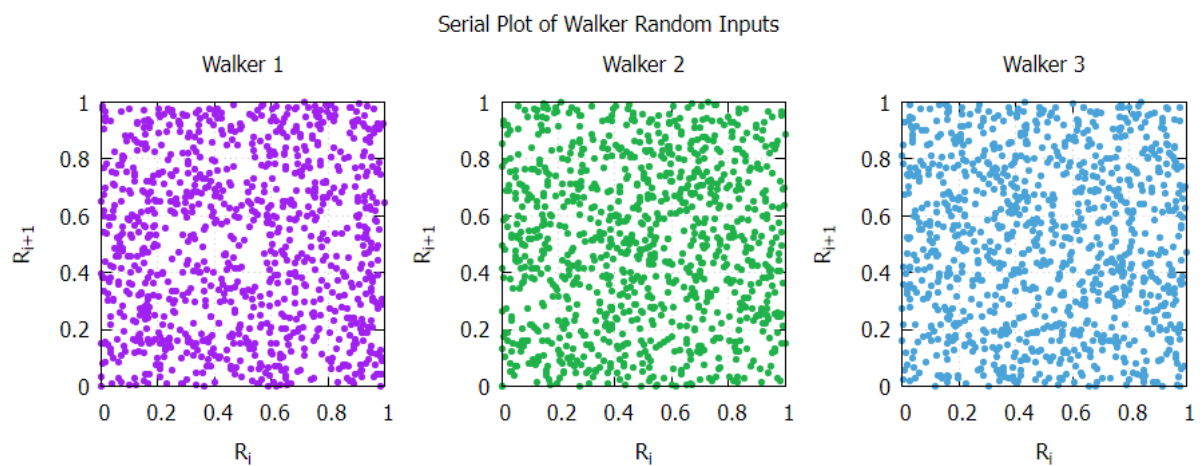
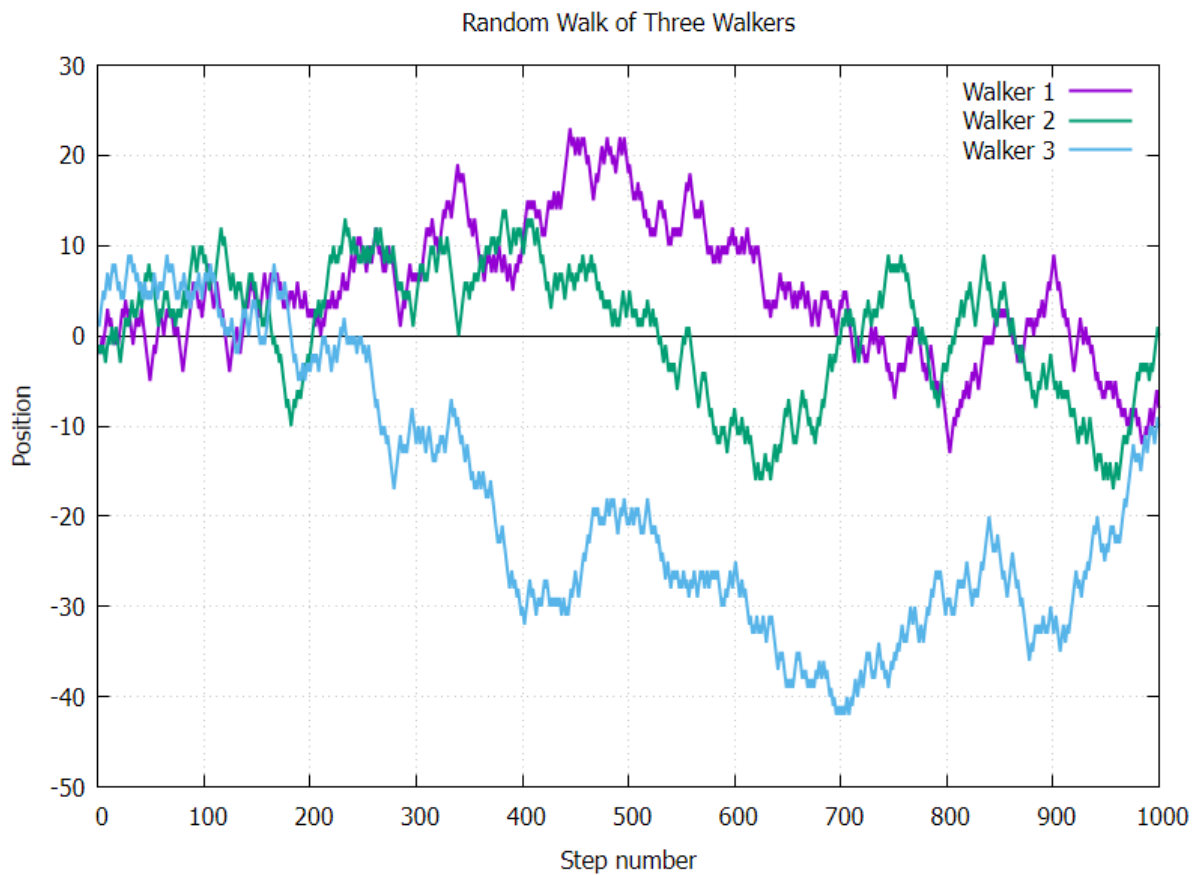
```

GNUPLOT code to generate the random walk plot -

```

1  set terminal pngcairo size 800,600
2  set output "random_walk.png"
3
4  set title "Random Walk of Three Walkers"
5  set xlabel "Step number"
6  set ylabel "Position"
7  set grid
8  set xzeroaxis linetype 1 linewidth 1 linecolor rgb "black"
9
10 p 'random_walk_data.txt' u 1:3 w l lw 2 title "Walker 1", 'random_walk_data.txt' u 1:5 w l lw 2 t

```



Problem 4:

For each value of step count N (Where $N_{max} = 20000$) –

1. Simulate an ensemble of $N_p = 500$ independent walkers starting at $x_0 = 0$.
2. Compute the ensemble averages over the 500 walkers:

Mean position: $\langle x_N \rangle = \frac{1}{N_p} \sum_{j=1}^{N_p} x_{N,j}$

Mean square position: $\langle x_N^2 \rangle = \frac{1}{N_p} \sum_{j=1}^{N_p} x_{N,j}^2$

Mean square displacement: $\Delta x^2 = \langle x_N^2 \rangle - \langle x_N \rangle^2$

3. Compare the numerical results against theoretical exact values:

$$\langle x_N \rangle = 0, \quad \langle x_N^2 \rangle = N, \quad \Delta x^2 = N$$

4. Plot $\langle x_N^2 \rangle$ and Δx^2 as a function of N to confirm the linear growth signature of normal diffusion.

Solution:

The result of this problem strongly depends on the input seed value. Here the Fortran code to solve this problem –

```

1  program main
2  implicit none
3  integer, parameter :: N_p = 500      ! Walkers
4  integer, parameter :: N_max = 20000 ! Maximum steps
5  integer, parameter :: N_step = 500
6  integer :: seed(4) = [20, 27, 91, 97]
7  integer :: i, j
8  integer :: position = 0
9  real :: random_num = 0.0
10 integer :: random_displacement_matrix(N_max,N_p+1) = 0
11 integer :: random_displacement_square_matrix(N_max,N_p+1) = 0
12 real :: statistical_matrix(N_max,4) = 0.0
13
14 do i = 1, N_max
15     random_displacement_matrix(i,1) = i
16     random_displacement_square_matrix(i,1) = i
17     statistical_matrix(i,1) = i
18 enddo
19 call srand(seed(2))
20 do i = 1, N_p
21     position = 0
22     do j = 1, N_max
23         random_num = rand()
24         if (random_num < 0.5) then
25             position = position - 1
26         else
27             position = position + 1
28         end if
29         random_displacement_matrix(j,i+1) = position
30         random_displacement_square_matrix(j,i+1) = position**2
31     enddo
32 enddo
33
```



```

34  do i = 1, N_max
35      statistical_matrix(i,2) = real(sum(random_displacement_matrix(i,2:N_p+1)))/real(N_p)
36      statistical_matrix(i,3) = real(sum(random_displacement_square_matrix(i,2:N_p+1)))/real(N_p)
37      statistical_matrix(i,4) = statistical_matrix(i,3) - statistical_matrix(i,2)**2
38  enddo
39
40  open(10, file='random_displacement_mean.txt', status='unknown')
41  do i = 1, N_max
42      write(10,*) statistical_matrix(i,:)
43  enddo
44  close(10)
45  call execute_command_line("gnuplot random_displacement_mean.gnu")
46  call execute_command_line("gnuplot random_displacement_mean_square.gnu")
47  call execute_command_line("gnuplot random_displacement_mean_square_displacement.gnu")
48  end program main

```

The GNUPLOT scripts to generate the plot of N vs $\langle x_N \rangle$, $\langle x_N^2 \rangle$, Δx^2 respectively are –

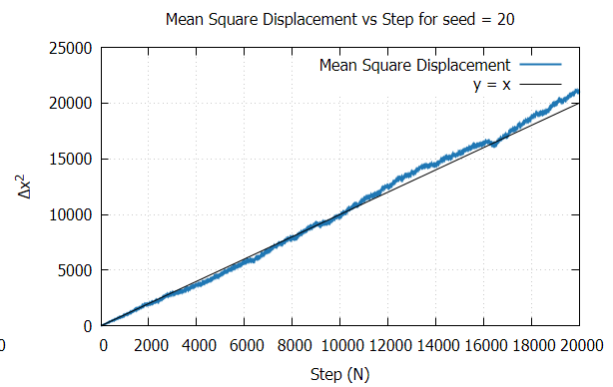
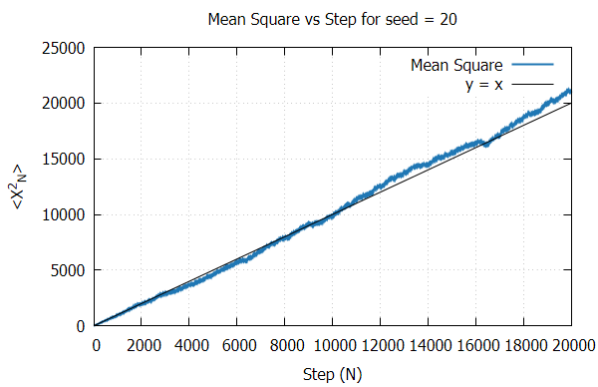
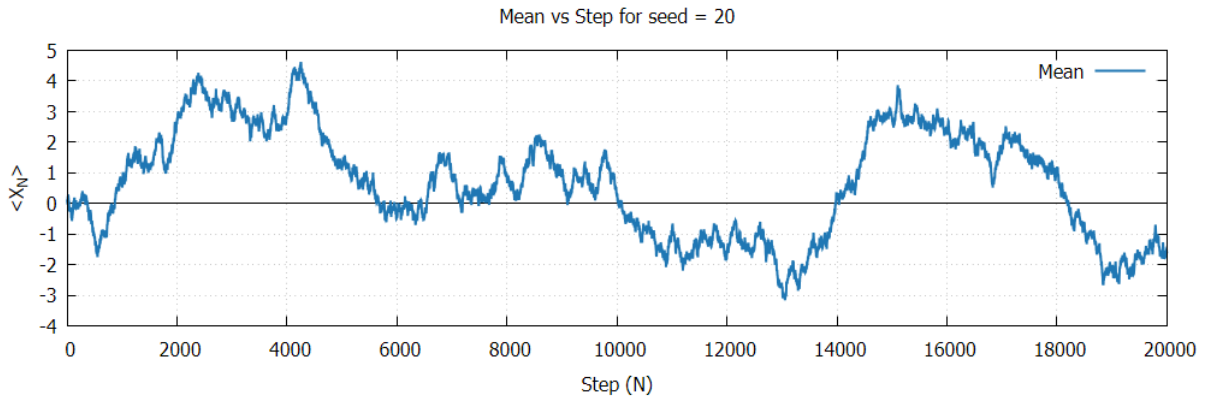
```

1  set terminal pngcairo size 1000,350
2  set output "random_mean_20.png"
3  set title "Mean vs Step for seed = 20"
4  set xlabel "Step (N)"
5  set ylabel "<X_N>"
6  set grid
7  set xzeroaxis linetype 1 linewidth 1 linecolor rgb "black"
8  plot "random_displacement_mean.txt" using 1:2 with lines lw 2 lc rgb "#1f77b4" title "Mean"

1  set terminal pngcairo size 600,400
2  set output "random_mean_square_20.png"
3  set title "Mean Square vs Step for seed = 20"
4  set xlabel "Step (N)"
5  set ylabel "<{X^2}_N>"
6  set grid
7  plot "random_displacement_mean.txt" using 1:3 with lines lw 2 lc rgb "#1f77b4" title "Mean Square
8                                     \,x lc rgb "#000000" title "y = x"

1  set terminal pngcairo size 600,400
2  set output "random_mean_square_displacement_20.png"
3  set title "Mean Square Displacement vs Step for seed = 20"
4  set xlabel "Step (N)"
5  set ylabel "{/Symbol D}x^2"
6  set grid
7  plot "random_displacement_mean.txt" using 1:4 with lines lw 2 lc rgb "#1f77b4" title "Mean Square
8                                     \Displacement", x lc rgb "#000000" title "y = x"

```



Comment: This simulation is done with pseudo-random number generator $seed = 20$. Theoretical expectation yields $\langle x_N \rangle = 0$. The simulated mean fluctuates boundedly within approximately $[-3, +5]$ around the zero baseline. The specific trajectory of this mean curve depends on the initial pseudo-random seed. Theoretical formula predicts $\langle x_N^2 \rangle = N$ (represented by the black reference line $y = x$). The simulated curve closely tracks the linear theoretical line. Due to $\langle x_N \rangle^2 \leq 25 \ll N$, the value of Δx^2 is nearly identical to $\langle x_N^2 \rangle$. It also closely tracks the linear theoretical line.
